



UNIVERSIDADE FEDERAL DE ALAGOAS

CAMPUS A.C. SIMÕES

IC - UFAL

LUCAS TRINDADE DE SOUZA

PROJETO JACKUT PROGRAMAÇÃO 2

MACEIÓ/AL

2026

LUCAS TRINDADE DE SOUZA

PROJETO JACKUT PROGRAMAÇÃO 2

Trabalho solicitado pelo professor Mario
Hozano, na matéria de
PROGRAMAÇÃO 2 (COMP 372), para
compor a nota AB2.

MACEIÓ/AL

RESUMO

Este trabalho aborda o desenvolvimento integral da lógica de negócio de uma rede de relacionamentos denominada Jackut, implementada na linguagem Java. Desenvolvido para a disciplina de Programação 2 (COMP 372), o projeto soluciona desafios de gerenciamento de contas, perfis dinâmicos, redes de amizades, interações públicas em comunidades e o processamento reativo de laços sociais complexos. A arquitetura baseia-se estruturalmente nos padrões Facade, Controller e Singleton, expandindo-se para a adoção combinada de Strategy e Factory Method com o intuito de erradicar o acoplamento condicional e maximizar o polimorfismo. A confiabilidade do software é assegurada pela persistência de dados via Serialização Binária nativa (Interface Serializable), que absorve a complexidade de modelos ricos e deleções em cascata preservando o encapsulamento estrito das classes. O design arquitetural é consolidado por um tratamento semântico rigoroso de exceções checadas, garantindo a aprovação e resiliência do sistema nos testes de aceitação automatizados através da ferramenta EasyAccept.

SUMÁRIO

RESUMO.....	3
1. Descrição:.....	5
DIAGRAMA:.....	6
2. Principais componentes e Interações:.....	7
2.1 Facade:.....	7
2.2 Controllers:.....	7
2.3 JackutRepository:.....	7
2.4 Modelos (Models) e Estratégias (Strategy):.....	7
3. Padrões de projeto adotados:.....	7
3.1 Padrão Facade:.....	8
3.2 Padrão Controller (GRASP):.....	8
3.3 Padrão Singleton:.....	8
3.4 Padrão Strategy:.....	9
3.5 Padrão Factory Method:.....	9
4. Teste da flexibilidade da arquitetura:.....	9
4.1 Isolamento de Atributos Dinâmicos:.....	9
4.2 Garantia de Comportamento FIFO Paralelo:.....	9
4.3 Modelos e Encapsulamento:.....	10
4.4 Preservação do Encapsulamento na Persistência:.....	10

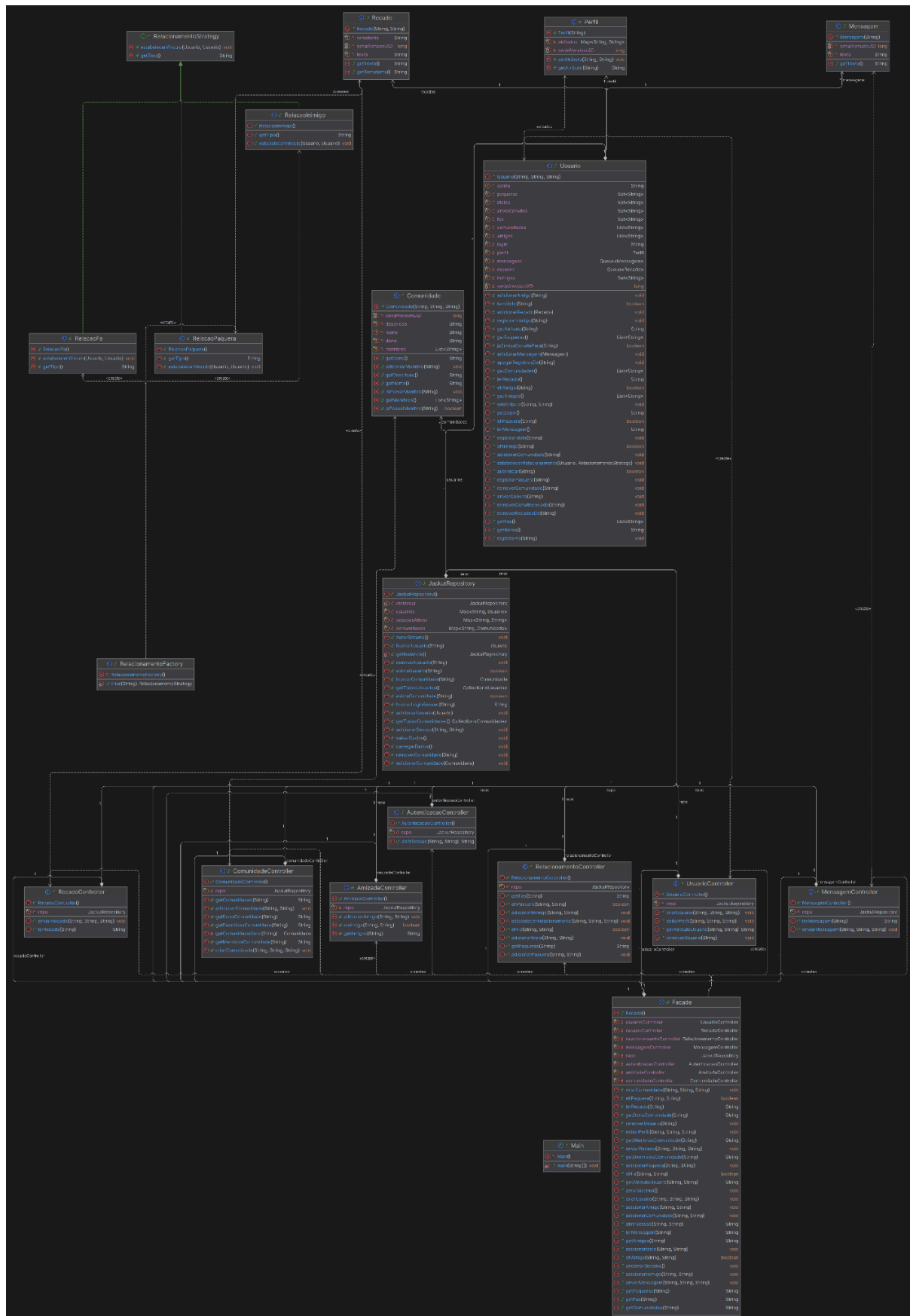
1. Descrição:

A arquitetura do projeto Jackut foi estruturada em três escopos principais: a Interface de Comunicação (isolada de forma estrita pela classe Facade), os Controladores de Regra de Negócio (como UsuarioController, ComunidadeController e RelacionamentoController) e as Entidades de Domínio (Usuario, Comunidade, Perfil, Recado, Mensagem e as Estratégias de Relacionamento).

A persistência de dados foi implementada localmente através da Serialização Binária nativa do Java (ObjectOutputStream e interface Serializable). Essa escolha arquitetural permitiu que as entidades mantivessem seus atributos rigorosamente encapsulados, utilizando modificadores final e blindando coleções internas contra acessos externos, além de garantir a persistência automática e íntegra de novos objetos complexos à medida que o sistema evoluiu.

Para orquestrar as complexas regras de negócio exigidas pelas novas User Stories (como a declaração de Ídolos, Paqueras e Inimigos, além da deleção em cascata de contas e o trânsito de mensagens em comunidades), o código adotou padrões de projeto comportamentais e criacionais. Esse refinamento garantiu o máximo aproveitamento do polimorfismo. Ademais, o tratamento de erros foi rigorosamente estruturado em um pacote de exceções personalizadas (Checked Exceptions, como InimigoException e RelacionamentoException), impondo um fluxo semântico às falhas e assegurando a conformidade exata com os scripts de teste da ferramenta EasyAccept.

DIAGRAMA:



2. Principais componentes e Interações:

O fluxo do sistema funciona da seguinte forma:

2.1 Facade:

Recebe as requisições do arquivo de teste EasyAccept e atua como despachante. Ela não contém regras de negócio, apenas direciona as chamadas para os métodos adequados dos Controladores e aciona o carregamento/salvamento dos dados no repositório.

2.2 Controllers:

As classes UsuarioController, AutenticacaoController, AmizadeController, RecadoController, ComunidadeController, MensagemController e RelacionamentoController concentram a orquestração do sistema. Elas validam as regras de negócio, coordenam as chamadas aos modelos e tratam os fluxos de erro antes de retornar os resultados para a Facade.

2.3 JackutRepository:

Atua como o banco de dados em memória e central de persistência. Ele interage com os Controllers fornecendo acesso seguro às coleções estruturadas (HashMaps de Usuários, Sessões e Comunidades). Na milestone 2, foi feito um refatoramento, em que o repositório, apenas métodos de delegação.

2.4 Modelos (Models) e Estratégias (Strategy):

Representam os dados de domínio e seu comportamento interno. Entidades como Recado e Mensagem atuam como objetos imutáveis. Entidades centrais como a classe Usuario operam sob encapsulamento estrito, possuindo responsabilidade ativa sobre a manipulação de suas listas (amigos, comunidades, fãs) e gerenciando deleções internamente, interagindo com estratégias (Relacionamento) injetadas para processar novos vínculos.

3. Padrões de projeto adotados:

A adoção de Padrões de Projeto (Design Patterns) no desenvolvimento do Jackut mostrou ser uma necessidade arquitetural para satisfazer requisitos não funcionais cruciais, como alta coesão, baixo acoplamento, extensibilidade e testabilidade. Com estes, a estrutura do projeto foi blindada contra quebras em cascata causadas por refatorações internas.

3.1 Padrão Facade:

- **Descrição Geral:** Padrão estrutural que fornece uma interface unificada e simplificada para um subsistema complexo. Ele atua como uma fronteira de proteção entre o cliente e as camadas internas da aplicação.
- **Problema Resolvido:** A biblioteca EasyAccept exige uma única classe de entrada. Sem a Facade, o cliente precisaria instanciar e coordenar todas as lógicas independentes (autenticação, amizades, comunidades).
- **Aplicação:** A classe Facade foi projetada de forma que ela fosse desprovida de qualquer regra de negócio, estrutura de decisão ou validação de dados. Sua única responsabilidade é receber as demandas do ambiente de teste e despachá-las para os controladores de domínio adequados (ex: AmizadeController, UsuarioController), garantindo que a interface pública do sistema permaneça imutável.

3.2 Padrão Controller (GRASP):

- **Descrição Geral:** Padrão arquitetural que atribui a responsabilidade de receber e coordenar as operações (eventos de sistema) a uma classe mediadora, separando a camada de interface das entidades de dados.
- **Problema Resolvido:** Para evitar o anti-padrão estrutural God Class. Se as entidades de domínio, como Usuario ou Comunidade, fossem encarregadas de realizar buscas no banco de dados, validar senhas e lançar exceções, elas violariam o Princípio da Responsabilidade Única (SRP), tornando a manutenção insustentável.
- **Aplicação:** Controladores segregados por domínio. O RelacionamentoController, por exemplo, tem a responsabilidade exclusiva de buscar os usuários no repositório e aplicar validações de amizade antes de consolidar o laço.

3.3 Padrão Singleton:

- **Descrição Geral:** Padrão criacional que garante que uma classe tenha apenas uma única instância em toda a execução do programa, fornecendo um ponto global de acesso a ela.
- **Problema Resolvido:** A fragmentação da lógica em múltiplos controladores gerou o risco crítico de múltiplas instanciações do banco de dados em memória (HashMap). Diferentes controladores operando em instâncias distintas do repositório causariam corrupção irreversível do estado da aplicação, falhas transacionais e perda de dados no momento da serialização.
- **Aplicação:** O JackutRepository utiliza um construtor privado e o método estático sincronizado getInstancia(), garantindo que a gravação do arquivo binário e a manipulação em memória ocorram em uma fonte única.

3.4 Padrão Strategy:

- **Descrição Geral:** Padrão comportamental que encapsula uma família de algoritmos, tornando-os intercambiáveis e isolando as lógicas de negócio do cliente.
- **Problema Resolvido:** O sistema evoluiu para suportar interações sociais com regras diametralmente distintas (como Fã/Ídolo, Paquera e Inimigo, incluindo efeitos colaterais complexos como o envio de mensagens automáticas em paqueras mútuas). Tratar essas variações proceduralmente, com extensas cadeias de switch ou instanceof dentro da classe Usuario, engessaria a arquitetura e dificultaria a manutenção.
- **Aplicação:** Foi elaborada a interface RelacionamentoStrategy e suas concretizações autônomas (RelacaoFa, RelacaoPaquera, RelacaoInimigo). A entidade Usuario passa a desconhecer a mecânica interna da relação, limitando-se a invocar o método estabelecerVinculo() da estratégia recebida. Essa solução atende rigorosamente ao Princípio Aberto/Fechado (SOLID), deixando o sistema aberto a novos tipos de laços sem necessidade de alterar classes consolidadas.

3.5 Padrão Factory Method:

- **Descrição Geral:** Padrão criacional que encapsula a lógica de instanciação de objetos.
- **Problema Resolvido:** Os comandos de teste chegam ao sistema na forma de texto puro (ex: a String "inimigo"). Era necessário traduzir esses textos para os objetos polimórficos do padrão Strategy sem poluir os controladores de negócio com lógicas criacionais ou estruturas de decisão condicionais repetitivas.
- **Aplicação:** A instanciação foi centralizada na classe utilitária RelacionamentoFactory. Ela recebe o identificador textual, processa a requisição e retorna a implementação correta de RelacionamentoStrategy. Ao isolar a factory de estratégias, os controladores mantêm-se focados exclusivamente em aplicar regras de negócio, preservando o baixo acoplamento.

4. Teste da flexibilidade da arquitetura:

As abordagens do sistema:

4.1 Isolamento de Atributos Dinâmicos:

A exigência de criar ou editar qualquer atributo não mapeado inicialmente foi solucionada utilizando a classe Perfil com um Map<String, String>, permitindo expansão sem alterar a classe base Usuario.

4.2 Garantia de Comportamento FIFO Paralelo:

Para as US4 e US7, que exigem a remoção cronológica sequencial, o uso de Queue combinado com o método .poll() fornece estruturamento adequado às caixas de entrada. A prova de flexibilidade na arquitetura se dá ao segregar totalmente as filas privadas (Recados) das públicas (Mensagens), sem que haja colisão entre elas.

4.3 Modelos e Encapsulamento:

A transição do sistema inicial(milestone 1) para o completo(milestone 2) fez o teste das defesas da arquitetura. Melhora no vazamento de coleções, já que as listas de Amigos, Membros de comunidades e Fãs passaram a retornar exclusivamente visualizações imutáveis (Collections.unmodifiableList). Além disso, a responsabilidade da "Deleção em Cascata" (US8) foi absorvida diretamente pelas próprias entidades, que passaram a apagar seus próprios vínculos, fornecendo um domínio robusto.

4.4 Preservação do Encapsulamento na Persistência:

A escolha inicial pela Serialização Binária absorveu de forma simples a complexidade crescente do sistema. Novos componentes, estratégias e comunidades foram persistidos sem a necessidade de reescrever código do Repositório, comprovando a resiliência das escolhas estruturais desde a Milestone 1.